

POSTS AND TELECOMMUNICATIONS INSTITUTE OF TECHNOLOGY

Department of Information Technology

INTELLIGENT SYSTEM DEVELOPMENT

ASSIGNMENT 02

From Data Representation to Deployable Intelligent Systems

Technical Report: Diabetes Screening, Real-Estate Valuation, and
E-Commerce Recommendation Prediction

GitHub repository: `minorin2847/INTEL_SYS_DEV_2026`
https://github.com/minorin2847/INTEL_SYS_DEV_2026

Student name: Bùi Nguyễn Hoàng Việt

Student ID: B23DCDT285

Class: E23CNPM02

Team: _____

Lecturer: Dinh Que Tran, Ph.D., Assoc. Prof.

Semester: I.2026

September 2026

Executive Summary and Objective

This report documents three end-to-end intelligent systems developed from Kaggle datasets. The common framework is *Data* $\rightarrow$ *Understand* $\rightarrow$ *Clean* $\rightarrow$ *Represent* $\rightarrow$ *Learn* $\rightarrow$ *Evaluate* $\rightarrow$ *Persist* $\rightarrow$ *Deploy*. The aim is not only to train accurate estimators, but to show precisely how a real observation becomes a reliable numerical input and then a usable web and mobile service.

Application	Task and dataset	Main representation	Selected result
Diabetes	Binary screening; CDC BRFSS 2015	Dense $X \in \mathbb{R}^{229781 \times 23}$	Random Forest; ROC-AUC 0.809, recall(positive) 0.735
House price	Regression; Vietnamese real-estate listings	Sparse $X \in \mathbb{R}^{201654 \times 92}$, $y = \log(1 + \text{price})$	Random Forest; RMSE 26,873 million VND, $R^2 = 0.186$
Customer behaviour	Recommendation classification; Sephora reviews	Sparse tabular $\oplus$ TF-IDF, $X \in \mathbb{R}^{104313 \times 28733}$	Logistic Regression; ROC-AUC 0.964, recall(not recommend) 0.876

Connection to Lecture 02: data representation

The real-world object is respectively a survey respondent, a property listing, and a product review. A CSV row is not yet a model input: it must become a feature vector $x = [x_1, \dots, x_d]^T$, while the dataset becomes $X \in \mathbb{R}^{N \times d}$. For the text application, a review may also be expressed as *text* $\rightarrow$ *tokens* $\rightarrow$ *token IDs* $\rightarrow$ *embeddings*, $E \in \mathbb{R}^{B \times T \times d}$. The deployed customer model uses a sparse TF-IDF representation, but the notebook demonstrates the embedding form required by the assignment.

Raw input $\rightarrow$ validation $\rightarrow$ same saved preprocessing $\rightarrow$ persisted model $\rightarrow$ JSON prediction
--

Data sources

Diabetes uses *Diabetes Health Indicators*; house price uses *Vietnamese Real Estate Listings*; and customer behaviour uses *Sephora Products and Skincare Reviews*. Their Kaggle links are provided in the final references. All experiments used Python 3.13, Windows 11, scikit-learn 1.9.0, and random seed 42.

Data Understanding and Representation Summary

Application	Raw record and target	Cleaning and encoding	Final model input
Diabetes	One BRFSS respondent; $y = \text{Diabetes_binary}$	23,899 duplicate rows removed; 8 continuous/count values scaled; 15 binary/ordinal values retained; 2 risk features engineered	$N = 229,781$, $d = 23$ dense float matrix. One request is 1×23 .
House price	One listing; $y = \text{price in million VND}$	Invalid price/area removed; median imputation plus missingness flags; skewed fields and target transformed by $\log(1 + x)$; seven categoricals one-hot encoded	$N = 201,654$, $d = 92$ sparse matrix; response is transformed back with $\exp(x) - 1$.
Customer behaviour	One Sephora review; $y = \text{is_recommended}$	Blank target rows removed; structured fields scaled/one-hot encoded; review title and body cleaned and joined; TF-IDF 1–2 grams	$d = 138$ tabular + 28,595 text features = 28,733 sparse features.

Feature meaning, types, and leakage controls

Diabetes begins with 21 health and demographic variables, including binary flags (e.g., high blood pressure), ordinal survey codes (general health, age, education, income), and continuous/count variables (BMI and unhealthy days). `TotalUnhealthyDays` and `CardioRisk` make the final 23 features. House price combines 11 numerical features (area, dimensions, rooms, access and missingness indicators) with seven categorical features such as property type, road type, province and parsed district. Customer behaviour combines profile/product fields with one textual review; rating and product-average rating are excluded because they reveal the recommendation outcome.

Why fitting belongs only to training data

The 70/15/15 split is stratified for diabetes; the customer split is grouped by author to reduce reviewer overlap. Imputers, scalers, encoders and the TF-IDF vocabulary are fitted only on training data. Re-fitting one of these objects on validation, test, or a user request would leak information and make a saved model inconsistent with the deployed service.

Mandatory representation dimensions

Here N is the number of cleaned observations and d is the number of numerical columns after feature engineering and encoding. In an embedding example, B is batch size, T is padded/truncated token length, and d is embedding width: the demonstrated review example uses $E \in \mathbb{R}^{1 \times 40 \times 16}$. Token IDs are integer vocabulary indices; embeddings are learned or assigned dense numerical vectors whose geometry represents word context.

Application 1 — Diabetes Screening

Problem and data. The task predicts whether a BRFSS respondent belongs to the pre-diabetes/diabetes target class. Thus X is the 21 survey features plus two engineered risk features and $y \in \{0,1\}$. The raw dataset has 253,680 rows and 21 input attributes; after removing exact duplicates it contains 229,781 observations with a 17.29% positive rate. This is a screening aid for prioritising confirmatory assessment, not a medical diagnosis.

Quality and preprocessing. Duplicate answer vectors were removed before splitting to prevent optimistic leakage. BMI tail values were retained: clipping or dropping them changed ROC-AUC by at most 0.003. Median imputation is inside the persisted pipeline. Ordinal codes remain monotonic numbers rather than being expanded to unnecessary dummies. The numerical representation is $X \in \mathbb{R}^{229781 \times 23}$; eight continuous/count columns use `StandardScaler`, while 15 binary or ordinal columns pass through unchanged.

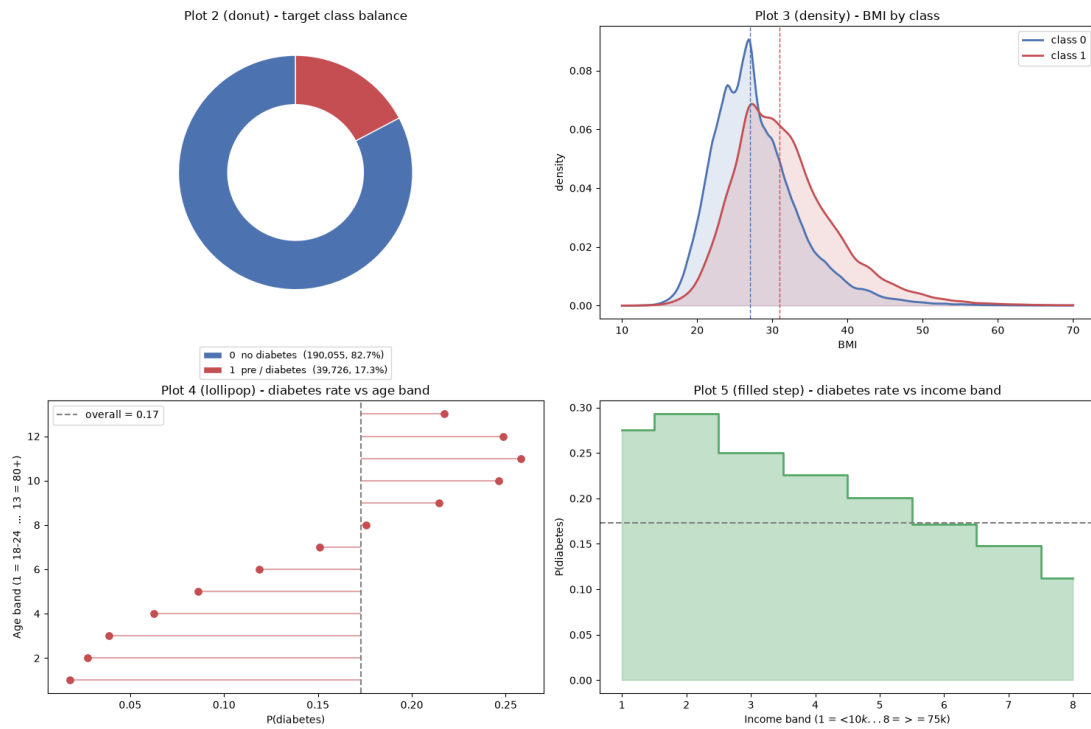


Figure. Diabetes EDA from the executed notebook: class balance, BMI by class, diabetes rate by age band, and income gradient.

Observation → interpretation → ML implication. The target plot shows a 17.3% positive class, so accuracy alone rewards a trivial negative predictor; stratification and recall are necessary. The BMI density for the positive class shifts upward but overlaps substantially with the negative class, so BMI is helpful rather than decisive. Diabetes rate climbs strongly across age bands while it falls by income band; preserving ordinal order and using a non-linear model is justified. The correlation analysis also identifies general health, blood pressure, cholesterol, walking difficulty, age and BMI as useful signals.

Models and final evaluation. Logistic Regression, Decision Tree, Random Forest, RBF SVM and KNN were compared with balanced class weights where applicable. Random Forest was selected: held-out accuracy 0.725, precision 0.357, recall 0.735, F1 0.480 and ROC-AUC 0.809. Its confusion matrix is $\begin{bmatrix} 20612 & 7897 \\ 1579 & 4380 \end{bmatrix}$. Missing a diabetic respondent (1,579 false negatives) is more serious than directing a healthy respondent to an additional test, so recall of the positive class is the key operational metric.

Application 2 — House-Price Prediction

Problem and data. The system estimates listing price from property, location and agent attributes. One row is a listing and y is a continuous, positive price in million VND, making this a regression task rather than a diabetes-style classification. The Kaggle source contains 236,226 listings; 201,654 remain after cleaning. Numeric fields include area, width, length, bedrooms, bathrooms, floors, alley width and agent listing count. Categorical fields include property type, position, direction, road type, province, agent role and district.

Cleaning and representation. Records with $\text{Price} \leq 0$, price above 200,000 million VND, or implausible area were removed. Bedrooms, bathrooms and floors are heavily missing, so each receives a missingness flag and the value itself is median-imputed inside the pipeline. The title is discarded because it can state the price. High skew motivates $\log(1+x)$ for selected numeric fields and for the target. Categorical columns use one-hot encoding with `handle_unknown=ignore` and a minimum frequency of 50. The result is sparse $X \in \mathbb{R}^{201654 \times 92}$ and $y = \log(1+\text{Price})$.

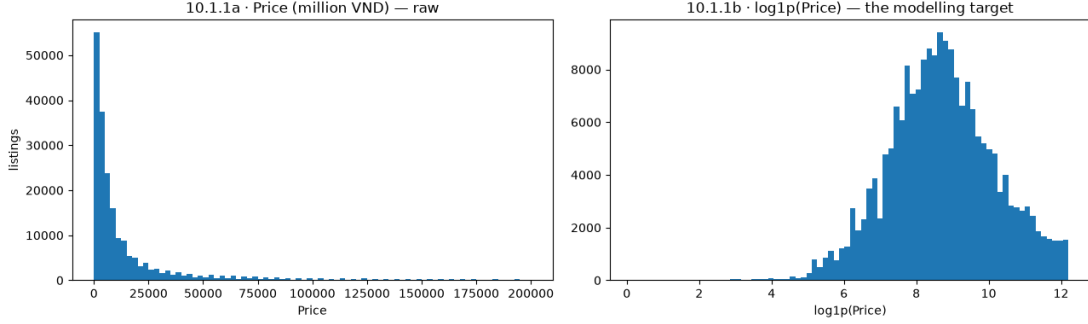


Figure. Raw and transformed price distributions. The log transform produces a much more usable modelling target.

Observation → interpretation → ML implication. Raw price is strongly right-skewed (skew 3.36), while the log target is close to symmetric (skew -0.07); modelling the transformed target prevents extreme prices dominating the loss. The pooled association between log area and log price is weak ($r \approx 0.017$), whereas province has $\eta^2 \approx 0.237$ and property type $\eta^2 \approx 0.120$. Area should therefore not be interpreted in isolation; a non-linear model can capture location–type interactions.

Models and final evaluation. Linear Regression, Ridge, Decision Tree, Random Forest and XGBoost were compared. Random Forest was selected because it achieved the strongest validation R^2 (0.183) and test result: MAE 10,626, MSE 722,149,435, RMSE 26,873 million VND and $R^2 = 0.186$. This modest fit is an honest limitation of weak signals, missing detail and coarse location, rather than a claim of precise appraisal.

Application 3 — E-Commerce Customer Behaviour

Problem and data. This supervised task predicts whether a Sephora skincare reviewer will recommend the product. A record is one review joined with product metadata, and $y = \text{is_recommended} \in \{0, 1\}$. The source joins `reviews_500-750.csv` and `product_info.csv` on product ID. From 116,262 raw reviews, 104,313 valid labelled reviews remain; 84.65% are recommendations. The business use is review-consistency QA, cold-start ranking from tabular fields, and category/brand merchandising.

Customer and text representation. Structured input contains profile values, product category and brand, price, popularity, ingredient/highlight counts and feedback measures. Six profile/category fields are one-hot encoded, continuous counts are scaled after log transforms where appropriate, and binary flags pass through. The text field is `review_title + ``.`` + review_text`; it is tokenised and transformed by TF-IDF over unigrams and bigrams. This produces 138 tabular features plus 28,595 text features. Token IDs denote positions in the learned vocabulary; the separate embedding demonstration maps 40 token positions to 16-dimensional vectors.

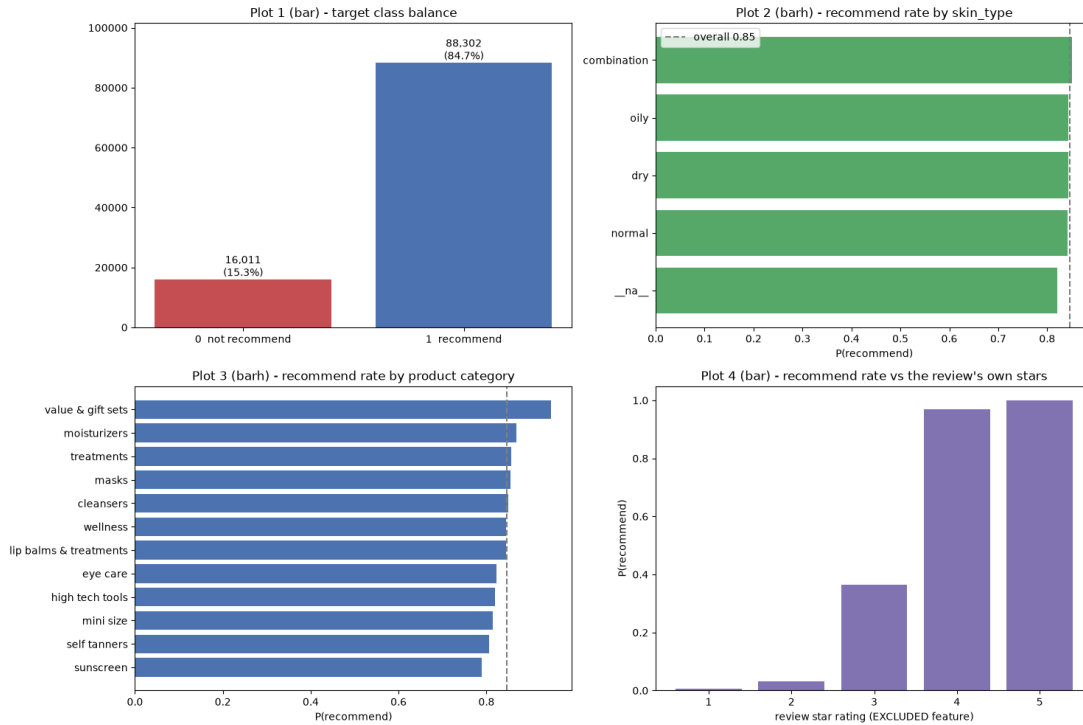


Figure. Customer-behaviour EDA: target balance, skin-type/category recommendation rates, and the excluded star-rating relationship.

Observation → interpretation → ML implication. Recommendations dominate, so the minority “not recommend” class needs class-aware evaluation. Category and profile rates vary less than the global rate, whereas a review’s own star rating almost deterministically reveals the target; rating and product-average rating are therefore excluded as leakage. Review text contains direct experience signals (e.g., irritation, repurchase intent, texture) and is expected to add far more information than the structured profile alone.

Models and result. Six alternatives were compared: tabular Logistic Regression, Decision Tree, Random Forest, HistGradientBoosting, Linear SVM, and text/combined linear models (SGD, Complement Naive Bayes, and Logistic Regression). Combined Logistic Regression won: test accuracy 0.918, macro-F1 0.858, ROC-AUC 0.964 and recall(not recommend) 0.876. Tabular-only ROC-AUC is 0.801, text-only 0.963, confirming that text materially improves the prediction but also makes this a retrospective use case.

Model Development, Evaluation and Selection

All estimators were trained behind the same train-fitted preprocessing pipeline. Evaluation used a held-out test set untouched during selection. For classification, precision measures the correctness of positive predictions, recall measures the fraction of the relevant class captured, F1 balances them, and ROC–AUC measures ranking quality across thresholds. For regression, $MAE = \frac{1}{N} \sum |y_i - \hat{y}_i|$, $MSE = \frac{1}{N} \sum (y_i - \hat{y}_i)^2$, $RMSE = \sqrt{MSE}$, and R^2 is explained variance.

Diabetes model	Accuracy	Precision	Recall	F1	ROC–AUC
Logistic Regression	.716	.352	.760	.481	.804
Decision Tree	.690	.328	.760	.459	.787
Random Forest	.727	.361	.748	.487	.810
RBF SVM	.707	.345	.776	.478	.800
KNN	.830	.550	.104	.174	.790

House-price model	Validation MAE	Validation RMSE	Validation R^2
Linear Regression	12,149	29,830	.028
Ridge	12,153	29,860	.026
Decision Tree	11,522	27,992	.144
Random Forest	10,811	27,349	.183
XGBoost	10,794	27,478	.176

Customer model	Representation	ROC–AUC	Macro-F1	Accuracy
Logistic Regression	tabular	.768	.622	.722
Decision Tree	tabular	.767	.637	.746
Random Forest	tabular	.810	.686	.806
HistGradientBoosting	tabular	.811	.664	.771
Linear SVM	tabular	.767	.622	.723
Logistic Regression	tabular + text	.966	.861	.919

Selection rationale. Random Forest provides the best recall/AUC trade-off for diabetes and robust non-linear handling for price. Logistic Regression is the appropriate customer final model because it works efficiently with 28k sparse features and permits exact coefficient-based explanations. The apparent customer text performance must not be overgeneralised: the review and recommendation checkbox are authored together, while post-publication engagement counts also limit prospective use.

Persistence, Inference, and Comparative Analysis

Persistence and deployment integrity

Each selected model is saved as `model/model_pipeline.joblib`, together with feature names and an input schema; the customer model also stores reference means for linear SHAP-style contributions. The FastAPI services load rather than retrain these artifacts. Reload tests verify that disk and in-memory predictions match. Consequently the same imputer, encoder, scaler, TF-IDF vocabulary and model are used during training, web inference and mobile inference.

Aspect	Diabetes	House price	Customer behaviour
Observation	Survey respondent	Property listing	Product review
Target	Diabetes class	Positive price	Recommendation class
Best model	Random Forest	Random Forest	Logistic Regression
Key metric	Recall(positive) .735	RMSE 26,873; $R^2 = .186$	Recall(not recommend) .876
Main quality risk	Duplicates; imbalance	Missing fields; target leakage	Rating/text label co-authorship
Most useful representation	23 numeric features	92 encoded features	28,733 sparse tabular + text features

Integrated discussion

The systems share train-only preprocessing, saved pipelines, input validation and REST inference, but their computational burdens differ. Diabetes is the easiest to deploy: it requires only 23 mostly numeric inputs and a small classifier. House price needs categorical handling and a target inverse transform; it is constrained by weak data signal. Customer behaviour is the most demanding because a request must generate a high-dimensional sparse text vector and explanation. Its structured, tabular-only form is a better choice for a truly pre-review decision; the combined form is valuable for retrospective review auditing.

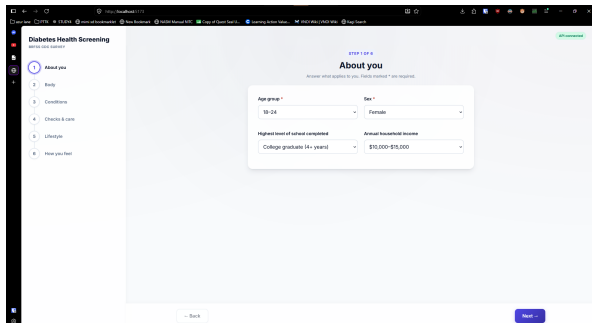
Architecture used by every application

Web or Flutter mobile UI → JSON POST `/predict` → Pydantic validation → loaded preprocessing pipeline → persisted estimator → JSON result and explanation

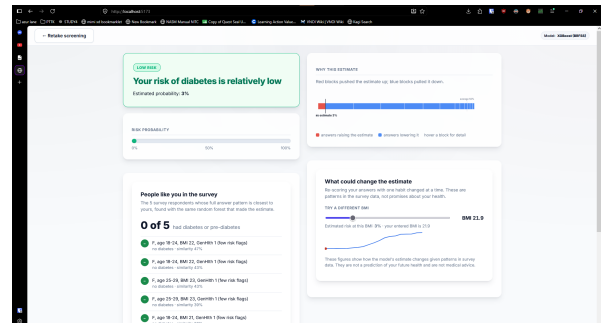
The response is a probability/confidence for the two classification systems and an estimated price for the regressor. The customer result also returns influential review terms and signed contributions; the diabetes and house-price interfaces present human-readable driver summaries. Training is deliberately separate from inference: a user request never calls `fit()`.

Web Deployment Evidence

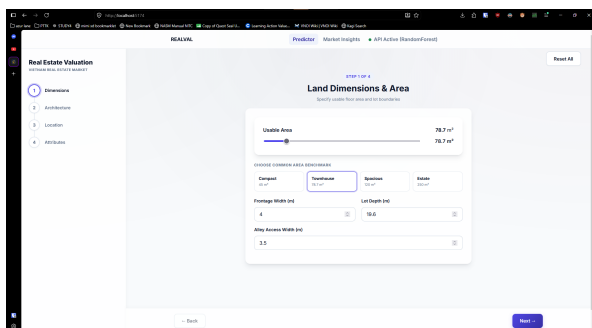
All web clients use React/Vite and communicate with a FastAPI `POST /predict` service. Forms mirror the saved input schemas: the diabetes wizard captures survey factors; the real-estate form captures area, dimensions and location; and the customer wizard captures profile, product and review text. The result screens display a prediction, probability or price, plus an explanation.



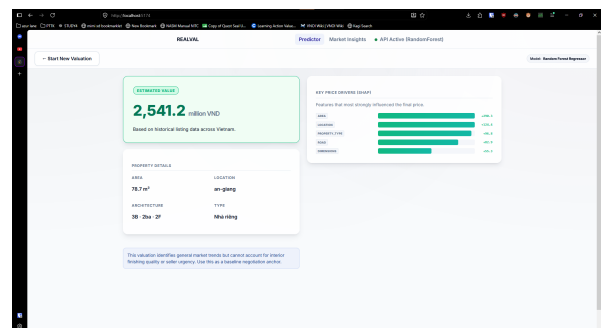
Diabetes web input



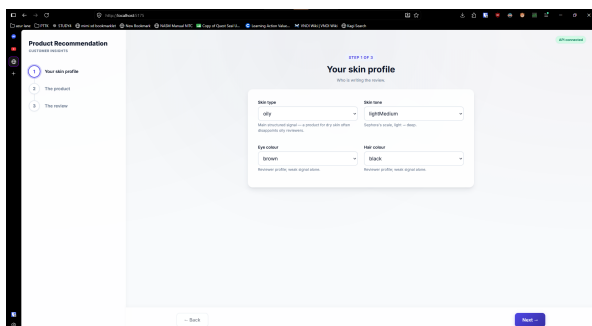
Diabetes web result: low-risk probability and factors



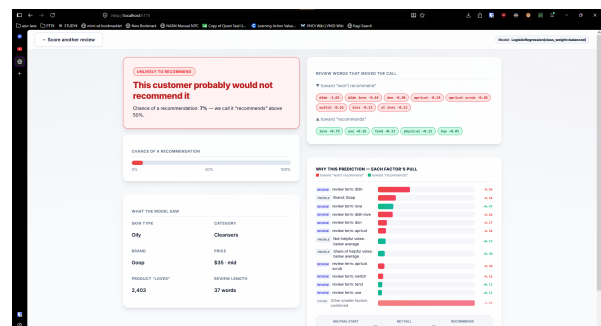
House-price web input



House-price web result: 2,541.2 million VND and SHAP drivers



Customer web input

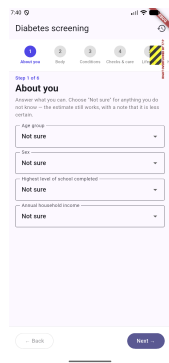


Customer web result: 7% recommendation probability and contribution chart

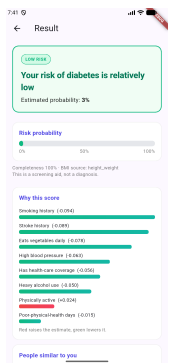
Valid-inference evidence. The web captures show completed form values, an API-connected status, and returned results. In the customer example, negative review terms such as “didn’t love” and “apricot scrub” pull the score down, while “love” pulls it up; this makes the output auditable rather than a bare label.

Mobile Deployment Evidence

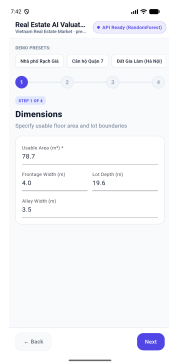
Each mobile client is a Flutter application that sends the same JSON field set to the corresponding REST endpoint. It does not train a model locally. The step-based forms validate user selections; returned predictions are rendered with a probability/valuation and an explanation. The API-connected/ready indicators in the captures provide runtime evidence of client-service communication.



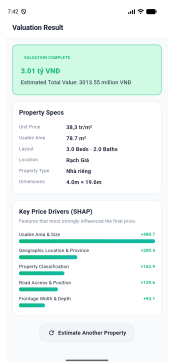
Diabetes input



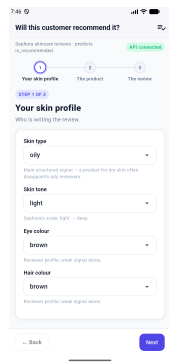
Diabetes result: 3% low risk



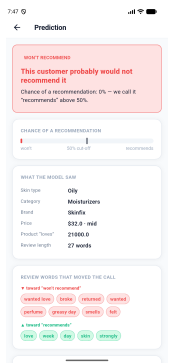
House-price input



House-price result: 3.01 billion VND



Customer input



Customer result: recommendation explanation

Mobile workflow. For example, the diabetes questionnaire groups user information into six steps; after submission, the REST response supplies the estimated risk, completeness cue and top factor directions. The property application returns unit price, total valuation and interpretable price drivers. The customer application returns the recommendation verdict, a 50% decision threshold and review terms that pushed the probability up or down.

Reproducibility, Limitations, and Future Work

Item	Diabetes	House price	Customer behaviour
Split	Stratified 70/15/15	70/15/15	Author-grouped 5-fold allocation
Saved artifact	Pipeline + schema	Pipeline + schema	Pipeline + vocabulary/means/schema
Deployment	FastAPI + React/Vite + Flutter	FastAPI + React/Vite + Flutter	FastAPI + React/Vite + Flutter
Reproduce	Notebook → artifact → API → web/mobile	Same workflow	Same workflow

The repository includes one notebook, model directory, API, web client, mobile client and requirements file for each application. A fixed random seed (42) is used for splitting and model construction. The house-price and Sephora source files are intentionally not committed because of their size; the Kaggle references in this report and the project README identify the required downloads. The README provides the run sequence: install requirements, execute the notebook to create the artifact, run Uvicorn, then launch the Vite or Flutter client.

Limitations

- **Diabetes:** survey responses are self-reported and the final recall of 0.735 is obtained with low precision (0.357). It is a screening support tool, not a diagnosis.
- **House price:** $R^2 = 0.186$ indicates the available listing fields cannot explain much price variation. Fine-grained geospatial and transaction data are needed before using it as a formal appraisal.
- **Customer behaviour:** review language is close to the recommendation decision, and engagement features are available only after publication. The combined result is therefore retrospective and the tabular-only score is the more realistic cold-start baseline.

Future work

Diabetes could use calibrated thresholds and explicit cost-sensitive decision analysis. House price could be improved with coordinates, price-per-square-metre comparables, sale dates and verified transactions. Customer behaviour could separate pre-review and post-review models, add aspect-level sentiment, and broaden the catalogue beyond skincare.

Conclusion

The central outcome of this assignment is that an intelligent system is more than a trained algorithm. Each completed application connects *raw data* → *clean data* → *numerical representation* → *model* → *evaluation* → *persisted service* → *user interface*. The three cases demonstrate why representation choices must follow the data: a compact dense health matrix, an imputed/encoded property matrix, and a sparse customer-plus-text matrix require different preprocessing, models and interpretation.

The most important technical challenge was maintaining one fitted transformation path from notebook to API. Saving the preprocessing and estimator together prevents a newly fitted scaler, encoder or vectoriser from changing the meaning of a request at deployment time. The main ML lesson is equally practical: accuracy alone can be misleading. Diabetes selection prioritises recall of an at-risk person; price estimation reports absolute error and weak explained variance; customer prediction highlights minority-class recall while stating the retrospective text limitation.

The deployment evidence shows that each selected artifact is usable from both a browser and a Flutter mobile application. With more time, the next improvement would be a shared internal library for schemas, feature construction and inference tests across the three systems, so the same representation contract is enforced once and reused everywhere.

Dataset and implementation references

1. A. Teboul, *Diabetes Health Indicators Dataset*, Kaggle: [alexteboul/diabetes-health-indicators-dataset](#).
2. qmanhbeo, *Vietnamese Real Estate Listings*, Kaggle: [qmanhbeo/vietnamese-real-estate-listings-may-2024](#).
3. nadyinky, *Sephora Products and Skincare Reviews*, Kaggle: [nadyinky/sephora-products-and-skincare-reviews](#).
4. Project notebooks, persisted pipelines, API clients, React/Vite applications and Flutter applications in the submitted Assignment 02 repository.

Raw Data → Clean → Represent → Learn → Evaluate → Persist → Deploy